

A packed example

Alright, we're out of space and still not past tagged unions. So let's make a leap and have a look at a fresh example packed with many Rust features that we didn't discuss:

```
struct AltUnder10(i32);

impl Iterator for AltUnder10 {
    type Item = i32;

    fn next(&mut self) -> Option {
        self.0 += 2;

        if self.0 < 10 {
            Some(self.0)
        } else {
            None
        }
    }
}

fn main() {
    let alt_numbers = AltUnder10(1);

    for num in alt_numbers {
        println!("{}", num);
    }
}
```

That looks like a giant leap. All it does is print the numbers 3, 5, and 7 in three lines (alternating numbers under 10, seeded with 1). Although the code showcases little that makes Rust unique, there is still a lot to explain that will help us get familiar with Rust.

Let's start with the *for* loop in *main()*, where all the action happens. It iterates through an object called *alt_numbers*, which we created just above the loop. In a previous example, we saw iterating through a list. But here the variable *alt_numbers* is an object of a struct *AltUnder10*, which we defined. How did it become iterable? Because we implemented the trait (think of interfaces in Java) called *std::iter::Iterator* for the struct.

This is what happens in the block that starts with *impl Iterator for AltUnder10*.

Any implementation of *Iterator* is required to have a method *next()* and an *associated type Item*. When called each time, *next()* should return a value of *Some*, or *None* if the iteration needs to stop.

Does our *next()* really return anything? In Rust, if you omit the semicolon for a statement inside a function, it acts as an implicit return. Another interesting thing is that even certain control flow structures like *if* are expressions in Rust (similar to Lisp).

Of course, our iterator needs to keep track of the state, and that's what the only field inside the struct *AltUnder10* is for. Note that the struct is declared as a tuple, meaning it has unnamed fields. This is why the field is referred to as *self.0*. If you declared the struct as *struct AltUnder10 { curno: i32 }* instead, you'll be referring to the field as *self.curno* inside the implementation. Creation of its variable inside *main()* will also require some change.

One last thing is about the input argument to *next()*. The name looks similar to the same thing in Python and the implicit *this* in C++. But what is *&mut*? There are two concepts here. The ampersand stands for a reference that does not change the ownership. Whoever calls *alt_numbers.next()* passes the reference to *alt_numbers* while still retaining the ownership. *mut* makes the reference mutable, without which *next()* won't be able to update the state of the object. So, finally, we're on to something that is related to ownership and memory management!

Syntax 'sugar' and zero-cost abstractions

When you are sure that an option value cannot be *None*, you can avoid matching on it and directly use the value by calling *unwrap()* on it (for example, *haystack.find(needle).unwrap()* gives you the position directly). But never ever misuse the feature. Matching on *None* will result in an abort. *unwrap()* should be used only when you are really sure, like inside an *if* block that already performs the necessary checks.

There is another syntax 'sugar' related to the option type that is safe but is for a different purpose. It is the question mark operator, which you can affix with every expression that would result in an option value to automatically return *None* when any of those values is *None*. As you might've guessed, this can only be used inside functions that return option types.

One last word is about performance. Hard core C programmers, even if they appreciate the beauty of these high-level features, will be reluctant to use them worrying about the performance penalty. That's where Rust's promise of zero-cost abstractions comes in. For example, if *T* is a reference type in *Option<T>*, Rust won't introduce any additional tag field, but simply use the NULL value to represent *None*. Yes, this is exactly what a C programmer does with pointers, but the point is, Rust keeps you away from the dangers of using NULL directly.

The official Rust documentation includes a benchmark result in which the use of an iterator is faster than loops. That's surprising and impressive. So you don't need to go back to low-level constructs and concepts unless you've proven the high-level ones slow or incompatible with the systems you want to deal with. **END** 🐧

 By: Nandakumar Edamana

The author has been involved with free software since his school days. A contributor to mainstream media since 2013, he has published three books in Malayalam. He was a software architect prior to joining one of the IITs as a PhD student.